.Net Cheat sheet (Entity-FrameWork Part)

Domain : Annotation for properties

prop string , min length 3, avec msg d' error
[MinLength(3, ErrorMessage = "invalide")]

prop phone number
[RegularExpression(@"^[0-9]{8}$)]

prop mail valid :
[DataType(DataType.EmailAddress)]

primary key :
[Key]
multiligne
[DataType(DataType.MultilineText)]

convert entities to DB tables : using dbset

Dbset entities => tables
public DbSet<Entity> table { get; set; }

OneToMany config :
Using Annotation



Entity1: prop… public (Entity2 primary key Type) ent2FK prop… **[ForeignKey("ent2FK")]** public virtual Entity2 ent2 { get; set; }	Entity2: public virtual IList<Entity1> ent1{ get; set; }

Using fluent Api

2 ways :
1/ configuration file => apply config
2/ context

1/ configuration file method :
step1
add entity Config File ent1config under infrastructure

```
public class Entity1config : IEntityTypeConfiguration<Entity>

public void Configure(EntityTypeBuilder<Entity> builder)
    {
 builder.HasOne(p => p.ent2 )
          .WithMany(p => p.ent1)
          .HasForeignKey(p => p.ent2FK)
}
```

step2:
apply configuration
go to OnModelCreating method in Context class

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfiguration(new ent1config ());
```

2/ config relation directly in OnModelCreating

```
        modelBuilder.Entity<Entity1>().HasOne(p => p.ent2 )
          .WithMany(p => p.ent1)
          .HasForeignKey(p => p.ent2FK)
```

/ no need to apply config

OneToMany config (rename associative table)

```
builder.hasmany(t=>t.entx)
        .withmany(p=>p.enty)
        .usingentity(p=>p.totable(“ name”))
```

complex type

Using Annotation

[Owned]

Entity3
string prop1
string prop2

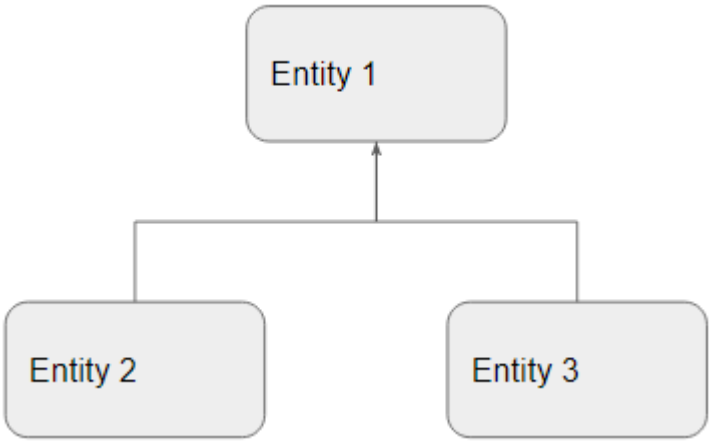
Entity4
Entity3 prop

Using fluent Api

```
modelbuilder.Entity<Entity4>.ownsone(p=>p.prop,x) {
x.property(p=>p.prop1)
x.property(p=>prop2)
}

;
```

Heritage configuration



TPH:Using discriminator

either Onmodel creating method or entity file configuration

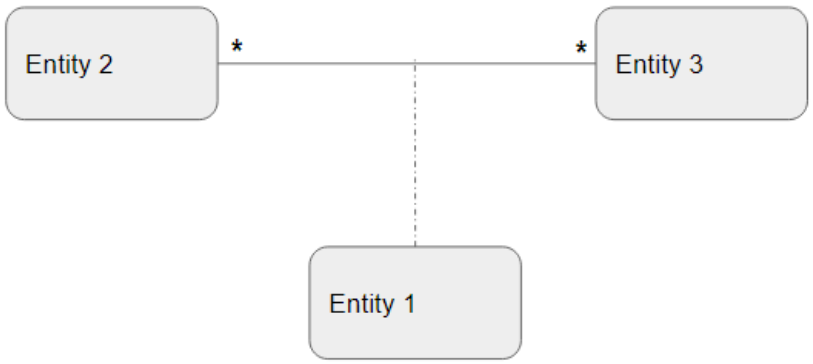
```
modelbuilder.Entity<Entity1>().hasdesciminator<type )>(“name”)
.hasvalue<>(entity1)(1)
.hasvalue<>(entity2)(2)
.hasvalue<>(entity3)(3)
```

TPT : table per type

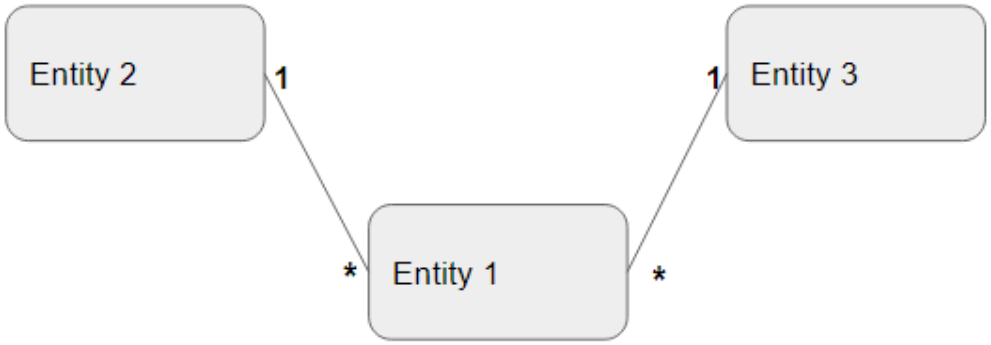
modelbuilder.Entity<Entity2>()totable(“ent2”)
modelbuilder.Entity<Entity>()totable(“ent3”)

Don’t forget to add new dbset for each entity

table porteuse



convert to two 1..* relations



step1 : config 2 OneToMany relations as previously mentioned
step 2 : primary key (2 foreign keys + prop)
step3:apply configuration

builder.haskey(r=>new{
r.ent1FK,
r.ent2FK
r.date}

design patterns

Interface

public interface IServiceEntity :IService<Entity>

service

public class ServiceEntity : Service<Entity>, IServiceEntiy